

Technik Programista

Programowanie – narzędzia

Agenci kodujący

Claude Code, Codex CLI, Gemini CLI

Instalacja, logowanie, podstawowe użycie i zasady pracy z LLM

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Co to jest i czym się różni od czatu	3
2. Wymagania wstępne (wspólne)	3
3. Claude Code (Anthropic).....	4
3.1 Instalacja	4
3.2 Logowanie	4
3.3 Podstawowe użycie	4
3.4 Plik CLAUDE.md	5
4. Codex CLI (OpenAI).....	5
4.1 Instalacja	5
4.2 Logowanie	5
4.3 Użycie.....	5
5. Gemini CLI (Google)	6
5.1 Instalacja	6
5.2 Logowanie	6
5.3 Użycie.....	6
6. Porównanie w skrócie	6
7. Jak z tego korzystać, żeby się czegoś nauczyć	7
7.1 Najpierw pytaj, potem każ pisać	7
7.2 Czytaj diff, zanim zatwierdzisz	7
7.3 Małe kroki	7
7.4 Każ sobie tłumaczyć	7
7.5 Testy pisze agent, ale Ty je uruchamiasz.....	7
7.6 Nie dawaj mu haseł	7
8. Wylogowanie na koniec zajęć	8
9. Zadanie na pierwsze zajęcia	8

1. Co to jest i czym się różni od czatu

Do tej pory pewnie używaliście LLM tak: wklejasz kod do okienka czatu, dostajesz odpowiedź, kopiujesz z powrotem. Działa, ale przy projekcie z 20 plikami jest męczące.

Agent kodujący działa inaczej. Uruchamiasz go **w folderze projektu** w terminalu. On sam:

- czyta Twoje pliki,
- edytuje je,
- uruchamia komendy (testy, `git`, `pip install`),
- pyta o zgodę, zanim zrobi coś większego.

Rozmawiasz z nim po polsku: „dodaj walidację do formularza rejestracji” – i on szuka pliku, robi zmianę, pokazuje Ci różnicę (diff).

Trzy najważniejsze narzędzia tego typu (stan: wrzesień 2026):

Narzędzie	Firma	Konto	Koszt
Claude Code	Anthropic	Claude Pro / Max / Team	płatny abonament
Codex CLI	OpenAI	ChatGPT Plus / Pro / Edu	płatny abonament (lub klucz API)
Gemini CLI	Google	zwykłe konto Google	darmowy limit (ok. 1000 zapytań/dzień)

💡 *Wszystkie działają tak samo z punktu widzenia użytkownika. Jeśli nie masz abonamentu – zacznij od Gemini CLI, bo jest za darmo. Umiejętność przenosi się 1:1 na pozostałe.*

2. Wymagania wstępne (wspólne)

1. **Git for Windows** – zainstalowany (patrz materiał o Git). Claude Code używa Git Bash jako terminala.
2. **Node.js** – potrzebny do Codex i Gemini (instalowane przez `npm`). Pobierz wersję LTS z <https://nodejs.org>.
3. Terminal: **PowerShell** (wpisz w Start: Terminal lub PowerShell). Prompt zaczyna się od `PS C:\`.

Sprawdzenie Node.js:

```
node --version
npm --version
```

3. Claude Code (Anthropic)

Dokumentacja: <https://code.claude.com/docs/en/quickstart>

3.1 Instalacja

PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Alternatywnie przez winget:

```
winget install Anthropic.ClaudeCode
```

Sprawdzenie:

```
claude --version
```

⚠️ Jeśli po instalacji terminal nie widzi komendy `claude` – zamknij terminal i otwórz nowy. Jeśli dalej nie widzi, instalator wypisał ścieżkę (zwykle `C:\Users\NAZWA\.local\bin`), którą trzeba dodać do PATH.

3.2 Logowanie

```
cd sciezka\do\projektu
claude
```

Przy pierwszym uruchomieniu otworzy się przeglądarka – logujesz się kontem Claude. Później już nie pyta. Zmiana konta: wpisz `/login` w sesji.

3.3 Podstawowe użycie

Po uruchomieniu `claude` piszesz normalnie:

```
co robi ten projekt?
gdzie jest obsługa logowania?
dodaj walidację e-maila w formularzu rejestracji
napisz testy dla funkcji oblicz_cene
zaktualizuj README o instrukcję uruchomienia
jakie pliki zmieniłem? zrób commit z opisowym komunikatem
```

Komenda	Co robi
<code>/help</code>	lista komend
<code>/clear</code>	wyczyść rozmowę
<code>/exit</code>	wyjście
<code>Shift+Tab</code>	przełącz tryb uprawnień (pyta / nie pyta o każdą zmianę)
<code>claude -c</code>	(z terminala) wróć do ostatniej rozmowy w tym folderze

3.4 Plik CLAUDE.md

W głównym folderze projektu możesz założyć plik **CLAUDE.md**. Claude Code czyta go na starcie każdej sesji. Wpisz tam to, co ma wiedzieć o projekcie:

```
# Projekt: sklep-django

- Django 5, SQLite, bez frontendu w React (na razie)
- Testy: pytest, uruchamiaj `pytest` przed każdym commitem
- Komentarze i nazwy zmiennych po polsku
- Nie instaluj nowych pakietów bez pytania
```

4. Codex CLI (OpenAI)

Repozytorium i dokumentacja: <https://github.com/openai/codex>

4.1 Instalacja

Przez npm (wymaga Node.js):

```
npm install -g @openai/codex
```

Lub instalator PowerShell:

```
powershell -ExecutionPolicy Bypass -c "irm
https://chatgpt.com/codex/install.ps1 | iex"
```

Sprawdzenie:

```
codex --version
```

4.2 Logowanie

```
cd sciezka\do\projektu
codex
```

Wybierz **Sign in with ChatGPT** – otworzy się przeglądarka. Wymaga planu ChatGPT Plus, Pro, Edu lub wyższego. Można też użyć klucza API OpenAI, ale wtedy płacisz za zużycie.

4.3 Użycie

Identycznie jak w Claude Code – piszesz, co ma zrobić:

```
wyjaśnij strukturę tego projektu
dodaj endpoint /api/produkty zwracający JSON
napraw błąd: formularz przyjmuje puste pola
```

Codex ma tryby zatwierdzania (approval modes) – domyślnie pyta przed edycją plików i uruchamianiem komend. Ustawienia trzyma w `~/.codex/config.toml`.

💡 *Odpowiednik CLAUDE.md to plik AGENTS.md w folderze projektu.*

5. Gemini CLI (Google)

Repozytorium i dokumentacja: <https://github.com/google-gemini/gemini-cli>

5.1 Instalacja

```
npm install -g @google/gemini-cli
```

Bez instalacji, jednorazowo:

```
npx @google/gemini-cli
```

Sprawdzenie:

```
gemini --version
```

5.2 Logowanie

```
cd sciezka\do\projektu
gemini
```

Wybierz **Login with Google** – logujesz się zwykłym kontem Google (tym od Gmaila). **Darmowy limit** to obecnie ok. 60 zapytań na minutę i 1000 dziennie – na zajęcia w zupełności wystarczy.

Alternatywa: klucz API z <https://aistudio.google.com/apikey> ustawiony jako zmienna `GEMINI_API_KEY`.

5.3 Użycie

```
opisz co robi ten projekt
dodaj model Zamowienie z polami: klient, data, kwota
napisz testy jednostkowe dla pliku utils.py
```

💡 *Odpowiednik **CLAUDE.md** to plik **GEMINI.md**.*

6. Porównanie w skrócie

	Claude Code	Codex CLI	Gemini CLI
Uruchomienie	<code>claude</code>	<code>codex</code>	<code>gemini</code>
Plik z instrukcjami	<code>CLAUDE.md</code>	<code>AGENTS.md</code>	<code>GEMINI.md</code>
Instalacja	<code>irm https://claude.ai/install.ps1 iex</code>	<code>npm i -g @openai/codex</code>	<code>npm i -g @google/gemini-cli</code>
Logowanie	konto Claude	konto ChatGPT	konto Google
Darmowe?	nie	nie	tak (z limitem)

Coraz częściej wszystkie trzy czytają też wspólny plik `AGENTS.md`, więc jeśli chcesz mieć jeden plik – zacznij od niego.

7. Jak z tego korzystać, żeby się czegoś nauczyć

Agent napisze kod za Ciebie. To nie jest problem. Problem jest wtedy, gdy **nie wiesz, co napisać**. Na rozmowie o pracę, na egzaminie i w pracy nikt nie zapyta „kto to napisał”, tylko „dlaczego to działa” i „co się stanie, jak zmienię tę linię”.

7.1 Najpierw pytaj, potem każ pisać

Zamiast: „zrób logowanie”.

Lepiej: „jak w Django działa logowanie? jakie pliki trzeba dotknąć?” -> dopiero potem „zrób to”.

7.2 Czytaj diff, zanim zatwierdzisz

Agent pokazuje, co zmieni. **Czytaj**. Jeśli czegoś nie rozumiesz – zapytaj: „co robi ta linia?”, „po co ten import?”, „czy da się prościej?”.

7.3 Małe kroki

Jedno zadanie = jedno polecenie = jeden commit. Nie: „zrób cały sklep”. Tak: „dodaj model Produkt” -> commit -> „dodaj widok listy produktów” -> commit -> ...

7.4 Każ sobie tłumaczyć

Po każdej większej zmianie: „wyjaśnij mi, co zrobiłeś, tak jakbym miał to opowiedzieć nauczycielowi”. To dosłownie to, co Cię czeka na następnych zajęciach.

7.5 Testy pisze agent, ale Ty je uruchamiasz

„napisz testy dla tego modułu” -> `pytest` -> patrzysz, co przechodzi, a co nie. Czerwony test to najlepsza okazja do zrozumienia kodu.

7.6 Nie dawaj mu haseł

⚠ **Nie wklejaj kluczy API, haseł do bazy, tokenów. Trzymaj je w .env, a .env w .gitignore.**

8. Wylogowanie na koniec zajęć

⚠ **Komputery w pracowni są wspólne. Agent po zalogowaniu trzyma token w Twoim profilu Windows – następna osoba będzie pracować na Twoim koncie i zużywać Twój limit.**

Narzędzie	Wylogowanie w sesji	Folder z tokenem (usuń, jeśli /logout nie działa)
Claude Code	<code>/logout</code>	<code>C:\Users\NAZWA\.claude</code>
Codex CLI	<code>/logout</code>	<code>C:\Users\NAZWA\.codex</code>
Gemini CLI	<code>/auth -> wyloguj</code>	<code>C:\Users\NAZWA\.gemini</code>

Do tego procedura z materiału o Git: usuń poświadczenie `git:https://github.com` z Menedżera poświadczeń Windows i wyczyść `git config --global user.name / user.email`.

9. Zadanie na pierwsze zajęcia

1. Zainstaluj **Gemini CLI** (albo inne, jeśli masz konto) i zaloguj się.
2. Wejdź do repozytorium `git-start` z poprzednich zajęć i uruchom agenta.
3. Napisz: „opisz, co jest w tym projekcie”.
4. Napisz: „dodaj do funkcje.py funkcję, która sprawdza, czy liczba jest pierwsza, i napisz do niej 3 testy w pliku `test_funkcje.py`”.
5. **Przeczytaj** wygenerowany kod. Zapytaj agenta o każdą linię, której nie rozumiesz.
6. Uruchom testy: `pytest`.
7. Załóż w projekcie plik `GEMINI.md` (lub `CLAUDE.md` / `AGENTS.md`) z trzema zasadami dla agenta.
8. `git add . -> git commit -m "Funkcja is_prime + testy (z pomocą agenta)" -> git push`.
9. Wyloguj agenta i usuń poświadczenia GitHub z komputera.